Otimização

5.3 Algoritmos genéticos

José Firmino Aguilar Madeira

De SA para AG: e se tivermos uma população?

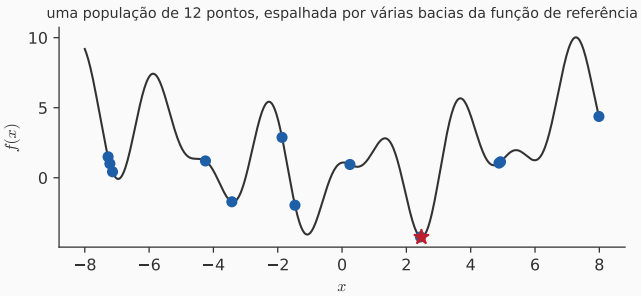


GLODS (5.1): vários pontos + pesquisas locais.

SA (5.2): um ponto que pode subir.

AG (5.3): uma população que evolui e troca informação.

De SA para AG: e se tivermos uma população?



GLODS (5.1): vários pontos + pesquisas locais.
SA (5.2): um ponto que pode subir.
AG (5.3): uma população que evolui e troca informação.

Pergunta: como fazer a população concentrar-se progressivamente nas regiões melhores sem perder diversidade demasiado cedo?

Esta pergunta contém praticamente todo o AG.

Ideia e vocabulário

Holland (1975), Goldberg (1989). A terminologia vem da genética; computacionalmente, temos soluções, variáveis, seleção, recombinação e perturbação.

Linguagem dos AG	Em otimização
indivíduo, cromossoma	uma solução $\mathbf{x}$ (codificada)
gene, alelo	variável (ou bit) e o seu valor
aptidão (*fitness*)	$F(\mathbf{x})$: maior = melhor
geração	iteração
seleção	escolher pais (prob. cresce com F)
cruzamento (*crossover*)	recombinar dois pais
mutação	perturbar um descendente

- A analogia é linguagem, não a razão matemática pela qual o método funciona.
- Só usa valores de f (ordem 0); contínuas, inteiras, binárias, permutações.
- Como o SA: heurística estocástica, sem garantia de ótimo global em tempo finito.

selecionar → recombinar → mutar → substituir

Codificação e aptidão

Binária (AG clássico)

$x \in [x^{(L)}, x^{(U)}]$ com ℓ bits: a cadeia $b_{\ell-1} \ldots b_0$ representa

$$x = x^{(L)} + \left(x^{(U)} - x^{(L)}\right) \frac{\sum_i b_i 2^i}{2^\ell - 1},$$

com resolução $(x^{(U)} - x^{(L)})/(2^\ell - 1)$. Para precisão ε_x: $\ell \geq \log_2\left(1 + (x^{(U)} - x^{(L)})/\varepsilon_x\right)$ (teto). Ex.: $[-5, 5]$, $\varepsilon_x = 10^{-4}$: $\ell = 17$. n variáveis: concatenar as n cadeias.

Real

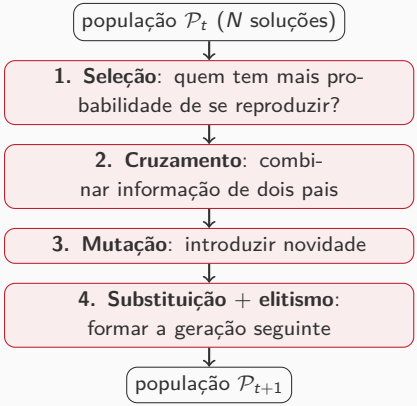
O cromossoma é o próprio vetor $\mathbf{x} \in \mathbb{R}^n$; os operadores trabalham com números reais. É o habitual hoje em otimização contínua (ga do MATLAB).

Aptidão: maximizar, com valores ≥ 0

Para $\min f$: $F = 1/(1 + f)$ se $f \geq 0$; ou $F = f_{\max} - f$ (na população); ou por ordenação (F = posição na lista ordenada, com o melhor a receber o maior F) — imune à escala de f e a valores extremos.

- Restrições: penalização (4.3.1) na aptidão, ou reparar/rejeitar os inadmissíveis (4.3.3).
- A codificação binária dá o exemplo mais transparente (a seguir); as suas dificuldades (precisão fixa, "penhascos de Hamming": $0111 \to 1000$ muda todos os bits) motivam a real.

Uma geração em quatro passos



A tensão de todo o AG

A seleção concentra a procura no que já parece bom; cruzamento e mutação geram novas soluções, e a mutação é uma fonte importante de diversidade.

$$\boxed{\textit{exploitation}} \longleftrightarrow \boxed{\textit{exploration}}$$

A mesma tensão da abertura do capítulo — agora gerida por operadores sobre uma população.

Seleção: quem se reproduz?

Roleta

Pai i com probabilidade

$$p_i = \frac{F_i}{\sum_j F_j}.$$

Sensível à escala: um indivíduo muito melhor que os outros fica com quase toda a roleta.

Torneio

Sortear s indivíduos (tipicamente 2) e ficar com o melhor. Só usa comparações; a pressão controla-se com s.

Elitismo

Copiar os e melhores diretamente para a geração seguinte: o melhor valor encontrado não se perde.

pressão de seleção demasiado forte $\rightarrow$ perda de diversidade $\rightarrow$ convergência prematura

Mais importante do que memorizar variantes.

Cruzamento e mutação: como criar novidade?

Cruzamento (com probabilidade $p_c \approx 0{,}6$–$0{,}9$)

Binário, a 1 ponto: cortar as duas cadeias na mesma posição e trocar as caudas, 100|01000, 000|10111 → 100|10111, 000|01000.

Real, BLX-β: $\mathbf{x} = a\,\mathbf{x}_a + (1-a)\,\mathbf{x}_b$, com $a \sim U(-\beta, 1+\beta)$ por coordenada — pode sair do segmento entre os pais.

Outras: 2 pontos, uniforme, aritmético, SBX.

Mutação (com probabilidade p_m por gene)

Binário: inverter o bit; $p_m \approx 1/(n\ell)$ ($n\ell =$ comprimento do cromossoma).

Real: somar $N(0, \sigma^2)$ a uma coordenada.

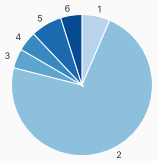
Permutações: trocar duas posições.

A mutação traz para a população valores que ainda não existiam nela; o cruzamento recombina os que já existem.

Uma geração à mão: min x^2 em $[-5, 5]$, 8 bits, $N = 6$, roleta, $p_c = 0{,}8$, $p_m = 0{,}05$

Aptidão $F = 1/(1 + x^2)$. População inicial aleatória:

i	cromossoma	x	$f = x^2$	F	p_i	acumulada
1	11010110	3,392	11,507	0,080	0,064	0,064
2	10001000	0,333	0,111	0,900	0,725	0,790
3	00010110	−4,137	17,117	0,055	0,044	0,834
4	00010111	−4,098	16,794	0,056	0,045	0,879
5	00101110	−3,196	10,215	0,089	0,072	0,951
6	11100100	3,941	15,533	0,060	0,049	1,000



A roleta: fatias proporcionais a F_i.

Uma geração à mão: min x^2 em $[-5, 5]$, 8 bits, $N = 6$, roleta, $p_c = 0{,}8$, $p_m = 0{,}05$

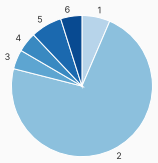
Aptidão $F = 1/(1 + x^2)$. População inicial aleatória:

i	cromossoma	x	$f = x^2$	F	p_i	acumulada
1	11010110	3,392	11,507	0,080	0,064	0,064
2	10001000	0,333	0,111	0,900	0,725	0,790
3	00010110	−4,137	17,117	0,055	0,044	0,834
4	00010111	−4,098	16,794	0,056	0,045	0,879
5	00101110	−3,196	10,215	0,089	0,072	0,951
6	11100100	3,941	15,533	0,060	0,049	1,000

Roleta: 6 números $\xi \sim U(0, 1)$ e o pai é o primeiro i com acumulada $\geq \xi$:

ξ	0,679	0,870	0,227	0,895	0,872	0,019
pai	2	4	2	5	4	1

O indivíduo 2 ($p = 0{,}73$) sai 2 vezes; o 3 e o 6 não são escolhidos.



A roleta: fatias proporcionais a F_i.

- Com $F = 1/(1 + f)$ o indivíduo 2 fica com quase toda a roleta: a roleta perde diversidade depressa. Torneio ou ordenação evitam isso.

A mesma geração: cruzamento e mutação — e o melhor piorou

Cruzamento a 1 ponto, pares (2,4), (2,5), (4,1); para cada par sorteia-se $\xi < p_c = 0{,}8$ e a posição do corte:

pais	cromossomas	corte	descendentes
2, 4	10001000, 00010111	corte após bit 3	10010111, 00001000
2, 5	10001000, 00101110	corte após bit 4	10001110, 00101000
4, 1	00010111, 11010110	corte após bit 7	00010110, 11010111

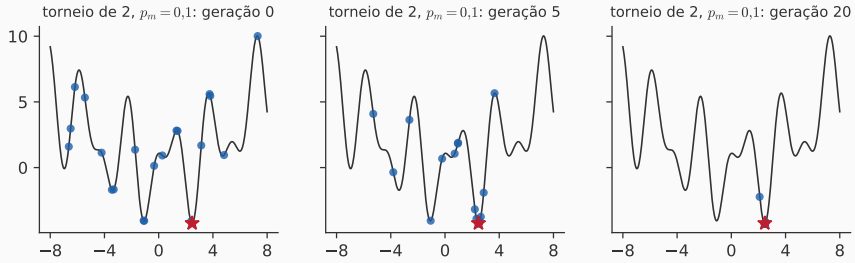
Mutação com $p_m = 0{,}05$ por bit (48 bits, esperam-se 2,4 inversões; saiu uma, a vermelho):

i	antes	depois	x	$f = x^2$
1	10010111	10010111	0,922	0,849
2	00001000	00001100	−4,529	20,516
3	10001110	10001110	0,569	0,323
4	00101000	00101000	−3,431	11,774
5	00010110	00010110	−4,137	17,117
6	11010111	11010111	3,431	11,774

Média de f: $11{,}9 \to 10{,}4$ — mas o melhor piorou ($0{,}111 \to 0{,}323$)

O indivíduo 2 foi "desfeito" pelo cruzamento. É exatamente para isso que serve o elitismo: copiar o melhor para a geração seguinte. Com elitismo e mutação que alcança todo X: garantia assintótica, em probabilidade (Rudolph, 1994); nenhuma em tempo finito.

Porquê uma população? Diversidade



A função de referência, $N = 20$, torneio de 2, BLX-0,5, $p_m = 0{,}1$: gerações 0, 5 e 20.

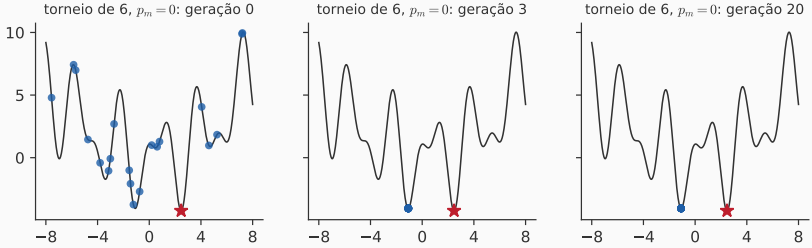
SA: uma trajetória tem de descobrir as bacias uma a uma.

AG: indivíduos diferentes ocupam regiões diferentes ao mesmo tempo.

Mas a seleção faz a população concentrar-se: na geração 20 estão todos na bacia do global. Correu bem — porque a diversidade durou o suficiente.

O recurso fundamental de um AG é a diversidade da população.

O perigo: todos iguais demasiado cedo



A mesma função, torneio de 6 (pressão forte) e sem mutação: em 3 gerações a população fechou-se na bacia de $x \approx -1{,}08$ (mínimo local).

Convergência prematura

População inicial espalhada → seleção forte → todos na mesma bacia. Sem diversidade, deixa de haver exploração global.

Mecanismos que a preservam:

- população maior;
- mutação;
- pressão de seleção moderada (torneio pequeno, ordenação em vez de roleta);
- *niching*, *fitness sharing* — só para saber que existem.

Algoritmo

AG: N, p_c, p_m, G gerações

1: gerar população inicial $\mathcal{P}_0$ de N indivíduos aleatórios; avaliar f (e F)
2: **for** $t = 0, \ldots, G-1$ **do**
3: **seleção**: escolher N pais de $\mathcal{P}_t$ (roleta/torneio)
4: **cruzamento**: aos pares, com prob. p_c, gerar dois descendentes
5: **mutação**: cada gene de cada descendente, com prob. p_m; projetar em X
6: avaliar os descendentes
7: **substituição**: fazer $\mathcal{P}_{t+1}$ igual aos N descendentes, com e deles substituídos pelos e melhores de $\mathcal{P}_t$ (elitismo)
8: **end for**
9: **return** o melhor de $\mathcal{P}_G$

- Custo: $n_f = N$ por geração; $n_f = N(G+1)$ no total. $N \approx$ 20–200.
- Paragem: G fixo; ou estagnação do melhor durante k gerações; ou diversidade da população abaixo de um limiar.
- Parâmetros a afinar: N, p_c, p_m, σ, s, e. Não há valores universais; p_m pequena e p_c alta são o ponto de partida habitual.

As N avaliações de uma geração são independentes

$\rightarrow$ paralelização: com N processadores e avaliações de custo semelhante, o tempo de uma geração pode aproximar-se do tempo de uma avaliação. Diferença prática face ao SA.

O que fazem os parâmetros?

parâmetro	pequeno	grande
N	pouca diversidade	mais avaliações por geração
p_c	pouca recombinação	muita mistura
p_m	perde novidade	quase pesquisa aleatória
pressão (s)	evolução lenta	convergência prematura
G (orçamento)	orçamento insuficiente	maior custo

Não existem valores universais. Ponto de partida habitual: p_c alta, p_m pequena, torneio de 2, elitismo.

Ao lado, N e orçamento mudam juntos; com $n_f = 420$ fixo: $N = 60$ ($G = 6$) 96%, $N = 6$ ($G = 69$) 46%.

Na função de referência, 300 corridas, sucesso $|x - x^*| < 0{,}3$:

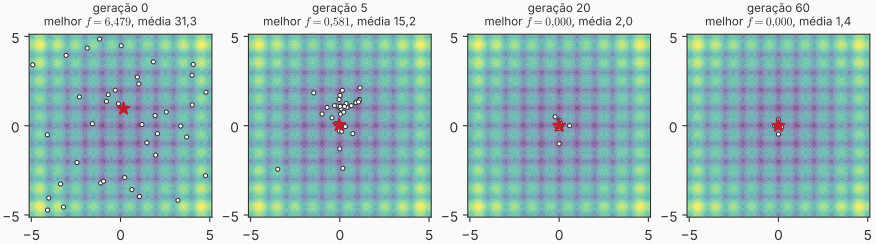
	sucesso	n_f
referência: $N = 20$, $s = 2$, $p_m = 0{,}1$	75%	420
pressão forte: $s = 6$	61%	420
sem cruzamento: $p_c = 0$	60%	420
população pequena: $N = 6$	34%	126
população maior: $N = 60$	96%	1260

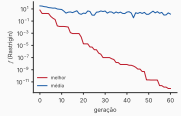
Neste exemplo pesa sobretudo N: uma população maior aumenta a diversidade inicial. p_m quase não muda o resultado ($p_m = 0$: 76%; 0,8: 78%) — com BLX-0,5, o cruzamento já gera novidade suficiente *neste problema*.

(A mesma configuração deu 71% em 100 corridas na tabela do capítulo: a taxa de sucesso também é uma estimativa.)

Rastrigin: explorar ou já convergir?

$f(\mathbf{x}) = 10n + \sum_i \left(x_i^2 - 10\cos 2\pi x_i\right)$, $n = 2$: 121 mínimos locais numa grelha, global em $\mathbf{0}$. $N = 40$, torneio de 2, BLX-0,5 ($p_c = 0{,}9$), mutação gaussiana ($p_m = 0{,}1$, $\sigma = 0{,}5$), 2 elites, 60 gerações (2440 aval.).

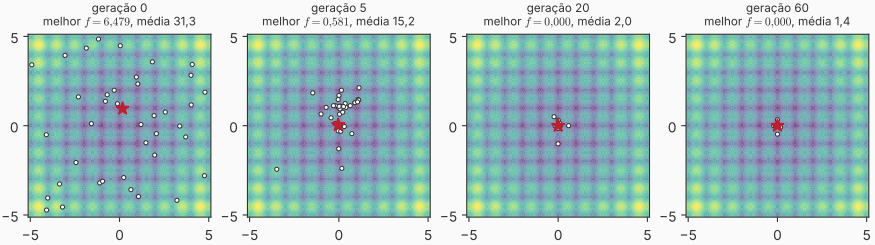


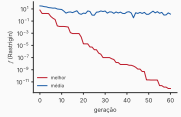


Pergunta: olhando para as gerações 0 e 5 — estamos a explorar ou já a convergir?

Rastrigin: explorar ou já convergir?

$f(\mathbf{x}) = 10n + \sum_i \left(x_i^2 - 10\cos 2\pi x_i\right)$, $n = 2$: 121 mínimos locais numa grelha, global em $\mathbf{0}$. $N = 40$, torneio de 2, BLX-0,5 ($p_c = 0{,}9$), mutação gaussiana ($p_m = 0{,}1$, $\sigma = 0{,}5$), 2 elites, 60 gerações (2440 aval.).





A população espalhada concentra-se na bacia do global por volta da geração 20 e depois refina: $f < 10^{-12}$. A média (azul) fica acima do melhor: a mutação mantém alguma dispersão.

O preço dos algoritmos genéticos

Vantagens

- Explora várias regiões em simultâneo.
- Representação flexível: contínuas, discretas, binárias, permutações.
- As N avaliações de uma geração são independentes: paralelizáveis.
- Extensão natural a vários objetivos.

Limitações

- Muitas avaliações; muitos parâmetros.
- Convergência prematura.
- Resultado estocástico.
- Precisão elevada é cara.
- Sem certificado de globalidade com orçamento finito.
- Não requer derivadas; com avaliações ruidosas, são precisos mecanismos adicionais para comparar indivíduos de forma fiável.

AG $\rightarrow$ método local

Na função de referência: 5 gerações ($n_f = 120$) põem o melhor a $|x - x^*| \approx 3 \times 10^{-3}$; uma pesquisa direta leva-o a 2×10^{-7} com 39 avaliações. Total 159, contra 420 de 20 gerações (uma corrida; em 300 corridas: 70% vs. 75%). O AG encontra a boa região; o método local refina.

GLODS, SA e AG: o que mudou?

	GLODS (5.1)	SA (5.2)	AG (5.3)
Estado	vários pontos	um ponto	uma população
Como escapa	*search* noutras regiões	aceita subidas	seleção + recombinação + mutação
Pesquisa local	pesquisa direta	não explícita (passeio aleatório)	nenhuma, salvo hibridação
Memória	lista de mínimos	corrente (+ melhor)	a população (e as elites)
Aleatório?	depende do *search*	sim	sim
Saída natural	vários mínimos	o ponto final (ou o melhor)	o melhor da população

Três respostas à mesma pergunta: como escapar de uma bacia local — e quanto custa.

No computador: implementação em MATLAB (codificação real)

O núcleo de `GeneticAlgorithm(f, n, lb, ub, N, G, pc, pm, sigma, e)`:

```
for t = 1:G
  [~, o] = sort(F);  E = P(o(1:e), :);      % elites
  i1 = randi(N,N,1);  i2 = randi(N,N,1);    % torneio
  pais = P(i1,:);  m = F(i2) < F(i1);
  pais(m,:) = P(i2(m),:);
  Q = cruzarBLX(pais, pc);                  % BLX-0.5
  M = rand(N,n) < pm;                       % mutacao
  Q = Q + M.*sigma.*randn(N,n);             % gaussiana
  Q = min(max(Q,lb),ub);  Q(1:e,:) = E;     % X; elites
  P = Q;  F = arrayfun(@(i) f(P(i,:)), 1:N)';
end
```

A função completa (`GeneticAlgorithm`, com o cruzamento BLX, a população inicial e `info.nfev`) está no código complementar da UC.

Chamada (Rastrigin, 30 corridas):

```
f = @(x) 20 + sum(x.^2 - 10*cos(2*pi*x));
for s = 1:30
  rng(s);
  [x, fx, info] = GeneticAlgorithm(f, ...
     2, -5.12, 5.12, 40, 60, 0.9, 0.1, 0.5, 2);
  fs(s) = fx;              % info.nfev = 2440
end
median(fs), prctile(fs, [25 75]), mean(fs < 1)
```

O MATLAB tem ga (Global Optimization Toolbox): `ga(f, n, [], [], [], [], lb, ub)`; aceita restrições e variáveis inteiras.

O MESMO EM PYTHON

`scipy.optimize.differential_evolution(f, bounds)` (evolutivo, com população) e `dual_annealing(f, bounds)`; `r.nfev` conta as avaliações. Não há AG clássico no SciPy.

E se não houver uma única forma de ordenar "melhor"?

Um objetivo: $f(\mathbf{x}_a) < f(\mathbf{x}_b)$ basta para selecionar.

E se não houver uma única forma de ordenar "melhor"?

Um objetivo: $f(\mathbf{x}_a) < f(\mathbf{x}_b)$ basta para selecionar.

Vários objetivos: $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_m(\mathbf{x}))$. Pode acontecer que, entre $\mathbf{x}_a$ e $\mathbf{x}_b$, nenhum seja melhor em todos os objetivos: $\mathbf{x}_a$ é melhor em f_1, $\mathbf{x}_b$ em f_2. Como selecionamos?

E se não houver uma única forma de ordenar "melhor"?

Um objetivo: $f(\mathbf{x}_a) < f(\mathbf{x}_b)$ basta para selecionar.

Vários objetivos: $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_m(\mathbf{x}))$. Pode acontecer que, entre $\mathbf{x}_a$ e $\mathbf{x}_b$, nenhum seja melhor em todos os objetivos: $\mathbf{x}_a$ é melhor em f_1, $\mathbf{x}_b$ em f_2. Como selecionamos?

Resposta (cap. 6)

dominância + diversidade — o NSGA-II (6.6) é um AG em que a seleção usa a ordem de dominância e uma medida de espaçamento (*crowding*).

A população, que aqui serve para explorar várias bacias, passa a servir para aproximar uma frente de Pareto inteira.

A mesma função, várias estratégias

A função de referência (1D, $[-8, 8]$, abertura do capítulo), 100 corridas com pontos iniciais aleatórios, sucesso $|x - x^*| < 0{,}3$:

	sucesso	n_f
Método local (1 arranque)	14%	36 [34; 38]
Multiarranque (8 arranques)	72%	290 [280; 299]
GLODS didático (determinístico)	sim	326
SA ($T_0 = 3$, $\alpha = 0{,}8$, $n = 10$)	33%	750 [671; 849]
AG ($N = 20$, $G = 20$)	71%	420

(n_f: mediana [Q1; Q3]; GLODS: uma corrida, que devolve a lista de 5 mínimos interiores; SA: ponto final, pontos iniciais aleatórios — no 5.2, sempre $x_0 = 6$: 60%.)

Rastrigin 2D (30 corridas, sucesso $f < 1$): Nelder–Mead $n_f = 90$ [85; 98] (mediana [Q1; Q3]), 3%; SA 5323 [4196; 6070], 97%; AG 2440, 100%.

Neste problema, com estes parâmetros — não é um ranking geral: os números mudam com o orçamento e os parâmetros.

O que se vê nos dois casos: escapar de uma bacia custa avaliações, e os estocásticos só se descrevem com várias corridas.

Quadro de consulta — capítulo 5

Método	Exploração global	Custo	Garantia	Aleatoriedade	Principal limitação
Multiarranque	um método local de vários pontos	uma pesquisa local por arranque	a de cada pesquisa local	a dos pontos iniciais	arranques na mesma bacia repetem trabalho
GLODS (5.1)	várias pesquisas diretas + *search*, reduzindo redundância por proximidade	$\leq 2n$ por *poll* + *search*; orçamento $N_{\max}$	estacionariedade (Clarke) dos limites refinantes, se f for Lipschitz e as direções refinantes densas (com $D_{\oplus}$ fixo: só nas de $D_{\oplus}$); lista de candidatos, sem garantia de encontrar todas as bacias	determinístico com *search* determinístico	bacias pequenas absorvidas pelo raio
SA (5.2)	um ponto que aceita piorar	uma avaliação por proposta; caro a T baixa	corrente: assintótica (espaço finito, arrefecimento logarítmico: impraticável); melhor visitado: como a pesquisa aleatória (3.2.1)	sim	T_0, α, n, σ sensíveis; precisão final
AG (5.3)	população, recombinação, mutação	$n_f = N(G+1)$; paralelizável	AG elitista: assintótica, em probabilidade, se a mutação alcança todo X (Rudolph, 1994); nenhuma em tempo finito	sim	muitos parâmetros; precisão custa gerações

Todos usam só valores de f (ordem 0), exceto o multiarranque, que herda a informação do método local.
Quando cada estratégia faz sentido: f suave com derivadas → cap. 3 (com multiarranque); caixa negra e poucos mínimos → pesquisa direta / GLODS; paisagem multimodal/rugosa ou variáveis discretas → SA / AG são alternativas naturais.
Para estudo. Na aula: as duas páginas seguintes. Rudolph, G., *Convergence analysis of canonical genetic algorithms*, IEEE Trans. Neural Networks 5(1):96–101, 1994.

O QUE DEVE FICAR — 5.3

1. **População:** o AG explora várias regiões em simultâneo.
2. **Seleção + cruzamento + mutação:** a seleção concentra (*exploitation*); cruzamento e mutação criam novidade (*exploration*). Seleção excessiva $\rightarrow$ perda de diversidade $\rightarrow$ convergência prematura.
3. **Custo:** N avaliações por geração — mas paralelizáveis.
4. **Estocástico:** várias corridas; sem certificado de ótimo global com orçamento finito.
5. **Hibridação:** o AG encontra a boa região; um método local refina.

A seguir: o capítulo 5 em resumo — e depois, com vários objetivos, o cap. 6.

O QUE DEVE FICAR — capítulo 5

Como escapamos de uma bacia local — e quanto nos custa fazê-lo?

1. Um método local tende a convergir para o mínimo da bacia de $\mathbf{x}_0$; procurar globalmente é explorar as outras: **GLODS**, vários pontos + pesquisas locais fundidas por proximidade (guarda as bacias que identifica); **SA**, um ponto que aceita piorar; **AG**, uma população que explora em simultâneo.
2. Escapar de um mínimo local custa avaliações de f: várias pesquisas, subidas aceites ou uma população.
3. Os métodos estocásticos devolvem uma variável aleatória: reportar $\geq$ 20–30 corridas, semente fixa, mediana e dispersão.
4. Sem hipóteses estruturais adicionais, não há garantia de identificar o mínimo global com orçamento finito (GLODS: estacionariedade, se f for Lipschitz e as direções refinantes densas; SA: ponto corrente só assintótica (espaço finito), melhor visitado como a pesquisa aleatória; AG elitista: assintótica, em probabilidade, se a mutação alcança todo X).
5. Um refinamento local final (cap. 3) reduz o orçamento para obter elevada precisão.

A seguir (cap. 6): e se houver duas coisas que queremos melhorar ao mesmo tempo?

Referências e recursos

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Holland, J. H., *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.
- Goldberg, D. E., *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.
- Madeira, J. F. A., *Optimização multi-objectivo da topologia de estruturas com algoritmos genéticos*, Tese de Doutoramento em Engenharia Mecânica, IST, Universidade Técnica de Lisboa, 2004 — em português.
- Rudolph, G., *Convergence analysis of canonical genetic algorithms*, IEEE Transactions on Neural Networks 5(1):96–101, 1994.

Recursos

- Código complementar: `GeneticAlgorithm` (cruzamento BLX, `info.nfev`). MATLAB: `rng(s)`; `ga` (Global Optimization Toolbox).
- Python: `scipy.optimize.differential_evolution`, `scipy.optimize.dual_annealing` (`r.nfev`).